Final Practice

Project Documentation

Xavier Canals Hernáez - 17/06/2025

Index

Implemented Sections.....3

1. Thread Pool.....3
2. Concurrent Scene Loading.....4
3. Concurrent Input Event Processing.....4
4. Concurrent Application of Geometric Transformations.....5
5. Concurrent Generation of Primary Rays.....5
6. Concurrent Intersection of Primary Rays.....6
7. Concurrent Framebuffer Update.....6

Implemented Sections

1. Thread Pool

The scripts **Engine_Thread_Pool.hpp** and **Engine_Thread_Pool.cpp** have been implemented within the **Engine** project.

This is a task-based thread pool that provides the following functions:

- **std::future< void > AddTask (Task newTask)**: Allows adding a new task to the internal task queue of the thread pool.

- **void Thread_function (std::stop_token stop_token)**: Responsible for executing the task assigned to each thread.

By working with **std::future**, this thread pool allows for assigning multiple tasks and waiting for their results using **future.wait ()**.

This thread pool is declared and initialized in the constructor of **Scene.hpp** within the **Engine** project using the **Start ()** method, and is gracefully shut down by calling **Stop ()**, also within the **Scene.hpp** class.

Additionally, a second thread pool has been created through the scripts **Ray_Tracer_Thread_Pool.hpp** and **Ray_Tracer_Thread_Pool.cpp** for the **Ray Tracer** project, intended for use in **Sections 5 and 6**. The key difference is that this second thread pool, instead of using a task queue, creates and launches all threads simultaneously, keeping them in a sleep state until a task is assigned. To use it, the **Run (Function_Type function)** method is called, which wakes up the threads and assigns them a specific function.

This thread pool includes a **barrier** to synchronize all threads, allowing the main thread to wait until all tasks are completed. It is designed to perform a single action concurrently and is declared and initialized in the constructor of **Path_Tracer.hpp** in the **Ray Tracer** project. To ensure proper cleanup, the **Stop ()** method is called within the destructor of the thread pool, avoiding the need for manual stopping.

2. Concurrent Scene Loading

Inside the **void load (Scene& scene)** function in **main.cpp** of the **App** project, sequential loading method calls have been replaced by three **AddTask** calls to the thread pool, allowing the threads to handle the loading concurrently. Afterwards, a **wait** is performed for each task so that the main thread waits until all content is fully loaded.

Using threads for this task introduced several **critical sections** where concurrent access could lead to bugs. To prevent this, multiple **mutexes** have been used to protect those areas:

- **entity_mutex** in the **create_entity** function
- **component_mutex** in the **create_component** function
- **create_form_mutex** in the **create** function

All of these methods are part of **Scene.hpp**.

3. Concurrent Input Event Processing

Within the **compute ()** function in **Input_Stage.cpp**, the code responsible for processing movement inputs has been removed, while the **scene-closing event** remains, as it doesn't require precise checking.

Instead, a **void prepare () override** method has been added. It runs once during the **run ()** function of the **Kernel** and launches a thread to execute **void check_input_state (EngineThreadPool& thread_pool)**. This function is **recursive**: it checks for keyboard events and relaunches itself after **10ms**. This way, a task for reading inputs is constantly enqueued, ready to run as soon as a thread is available.

Additionally, **std::this_thread::yield ()** is used in the **main thread** to give priority to other threads, reducing unnecessary CPU usage.

To ensure proper cleanup, a **bool finish** flag has been added in the **cleanup ()** method of **Input_Stage.cpp**, preventing further task creation.

4. Concurrent Application of Geometric Transformations

To implement this section, the **update_component_transforms** function in **Path_Tracing.cpp** was modified. The initial **for** loop managing the **camera transform** remains unchanged, as concurrent processing is deemed unnecessary for typically single-camera scenes.

However, the second loop, which handles **model transforms**, was replaced with thread pool calls. Initially, the idea was to assign **three separate threads** to update **position**, **rotation**, and **scale** respectively—but this was dismissed as too costly, given the minimal workload of each update.

Instead, **one thread per model** was used. As an optimization, models are processed in **blocks of 100 per thread**. For small scenes like the current one, the improvement is negligible, but this setup prepares the system to efficiently manage **thousands of models concurrently**, each thread handling a block of 100.

The **main thread** waits for all worker threads to finish using **future.wait ()**.

5. Concurrent Generation of Primary Rays

Building on the strategy used in Section 4, the **calculate ()** function in **Pinhole_Camera.cpp** was updated to divide the image into smaller blocks and assign their generation to the thread pool, rather than generating all rays sequentially.

The **second thread pool** (discussed in Section 1) is used here. It consists of **sleeping threads** waiting to be assigned a parallel task.

The image is divided by **rows**, with each thread responsible for generating a **subset of rays** based on its index, determining the range of lines to process.

As mentioned, the **thread pool itself handles synchronization** and ensures that the **main thread** waits until all **primary rays** are generated.

6. Concurrent Intersection of Primary Rays

For this section, the **primary ray buffer** is divided into **32x32 square tiles**, each assigned to a thread. Each thread is responsible for intersecting the rays in its tiles with the scene and **accumulating lighting data** into the **framebuffer** concurrently.

Threads may handle **one or more tiles**—for example, processing tiles **0, N, 2N, etc.** which balances the workload and prevents threads with more complex tiles (e.g., more geometry or lighting) from taking significantly longer.

As in Section 5, the **second thread pool** is used.

7. Concurrent Framebuffer Update

For the final section, a strategy similar to **Section 3** (input event management) was used. The call to **refresh_framebuffer** in **Path_Tracing.cpp** was moved from the **compute ()** function to **prepare ()**, which runs once. This method launches a task to update the framebuffer and schedules the same task to rerun every **~40ms** (approx. **25fps**). This ensures periodic framebuffer updates **independent from primary ray processing**.

To prevent **race conditions** when accessing the path tracer's **snapshot**, a **snapshot_mutex** was added to protect the critical section and avoid visual bugs or artifacts.

Lastly, a **bool finish** flag was introduced in the **cleanup ()** method of **Path_Tracing.cpp** to prevent continued creation of these tasks after shutdown begins.